

The CNN Anomaly Detector, End to End

A learning resource for multivariate time-series anomaly detection on batch-distillation sensor data.
Current result: validation F1 $\approx$ 0.60, competition-portal F1 $\approx$ 0.63.

This document explains how the system works so you can reason about its behaviour and improvements yourself — not just raise a number. Every claim is tied to the actual code in `src/`.

1. The problem and the data

You are detecting anomalies in an industrial batch-distillation process instrumented with 18 sensors (temperatures, pressures, flows, levels — tags like T701, PDI702, FT703). The defining constraint is that **the training data has no anomaly labels**. You get 28 unlabeled training runs, 10 labeled validation runs (per-timestep 0/1 labels), and 53 unlabeled test runs scored by an external portal.

That constraint forces the entire design. You cannot train a classifier that learns 'anomaly vs normal', because you have no anomaly examples to learn from. So you take the **reconstruction / novelty** route: teach a model what NORMAL looks like, then flag whatever departs from it. This is why an *autoencoder* is the natural choice — it is trained only to copy normal data through a bottleneck, and it reconstructs unfamiliar (anomalous) patterns badly. The reconstruction error becomes the anomaly score.

2. The pipeline at a glance

Every one of the three pipelines (`run_cnn.py`, `run_hybrid.py`, `run_final_hybrid.py`) follows the same spine. The CNN pipeline is the model of record:

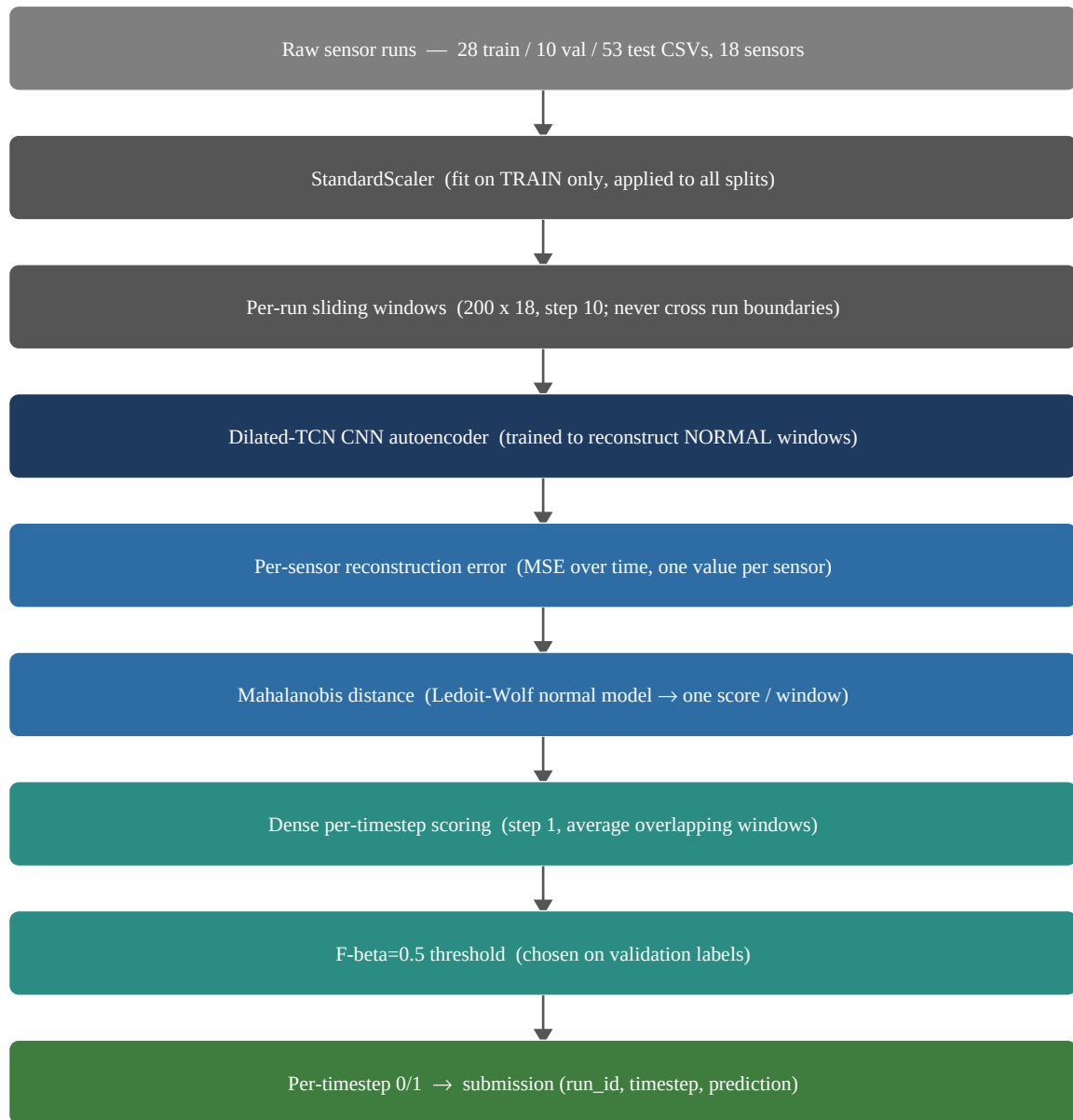


Figure 1. End-to-end methodology. Grey = data handling, navy = the model, blue = scoring, green = output. The validation labels are used at two points only: to pick the best training checkpoint and to choose the threshold — never to train the network.

3. The pipeline, step by step (with the 'why' and the 'what if')

3.1 Ingestion and per-run loading

`load_split_per_file` loads each run as its own DataFrame and keeps them *separate*. This matters for the very next steps: runs are independent experiments, and a window must never span two of them.

3.2 Scaling — `StandardScaler`, fit on train only

Sensors live on wildly different scales (a temperature in the hundreds, a pressure difference near zero). A neural network trained on raw values would let the large-magnitude sensors dominate the loss. `StandardScaler` converts every sensor to mean 0 / std 1. **Why fit on train only?** The scaler's mean and std are parameters learned from data; fitting them on validation or test would leak information about the evaluation set into preprocessing. **What if you skipped it?** The MSE loss would be dominated by a few

high-variance sensors and the autoencoder would essentially ignore the rest — many real anomalies would become invisible in the score.

3.3 Windowing — 200 timesteps, per run

The CNN does not see one timestep at a time; it sees a **window** of 200 consecutive timesteps across all 18 sensors (`create_windows_per_file`, `WINDOW_SIZE=200`). Training uses `stride STEP=10` (windows every 10 steps — enough coverage without 20x redundancy); validation/test use `EVAL_STEP=1` for a dense, per-timestep score.

- **Why 200?** The project's window-size tuning (notebook 04) found F1 kept rising as the window grew from 20 to 200 — the anomalies here are slow, sustained drifts, not sharp one-step spikes, so the model needs a wide temporal context to recognise 'this pattern is abnormal'.
- **Why per-run (never splicing runs together)?** An earlier bug concatenated all runs into one array before windowing, so a window could stitch the end of one run onto the start of an unrelated one — a fake transient the model wasted capacity 'explaining'. Fixing this was one of the six original bug fixes.
- **What if you made it 400?** More context, but two costs: fewer training windows, and short runs (one train run is only 743 timesteps) would yield very few or zero windows — you would silently drop data. 200 is a sweet spot given the run lengths.

3.4 The model — a dilated-TCN autoencoder

The network (`ConvAutoencoder1D`) is a 1-D convolutional autoencoder built from dilated residual blocks. Section 4 dissects it layer by layer. The one-line intuition: it *compresses* a 200-step window down through a bottleneck and then *rebuilds* it, and because it was only ever trained on normal windows, it can only rebuild normal-looking patterns well.

3.5 Training — reconstruct, and keep the best checkpoint

Training minimises mean-squared reconstruction error (`nn.MSELoss`) with Adam (`lr=1e-3`) for 50 epochs. Crucially, every 5 epochs it scores the validation set and **keeps the best-scoring checkpoint**, not the final epoch:

This exists because the training loss falls smoothly for all 50 epochs (`0.187 → 0.007`) while validation F-beta peaks early and then *drifts down*. In your last GPU run the best checkpoint was **epoch 45 (0.601)** and epoch 50 scored only 0.558. Without checkpoint selection you would ship the worse, over-trained epoch-50 model. This is the autoencoder slowly learning to reconstruct anomalies too (because the unlabeled train set is not guaranteed anomaly-free), which shrinks the very normal-vs-anomalous error gap the score depends on.

What if you trained to convergence with no checkpointing? You saw it: ~0.05 F1 lower. **What if you evaluated every epoch instead of every 5?** Finer selection, marginally better, but 5x the validation-scoring cost during training.

3.6 Scoring — per-sensor error, then Mahalanobis distance

This is the single most important scoring decision in the project. A naive score averages the reconstruction error over every sensor AND timestep into one number. That is exactly wrong here: a genuine alarming spike on one normally-precise sensor gets diluted by another sensor's ordinary noise.

- 1 **Per-sensor error** (`get_reconstruction_errors(per_sensor=True)`): average the error over time only, giving one error value *per sensor* per window — an 18-dim error vector, not a scalar.

- 2 **Fit a normal model** (`fit_mahalanobis`): on windows known to be normal, estimate the mean error vector μ and the inverse covariance precision using a Ledoit-Wolf shrinkage estimator (well-conditioned even with few samples).
- 3 **Mahalanobis distance** (`mahalanobis_scores`): score each window by how far its error vector sits from the normal cloud, weighting each sensor by how much it *normally* varies and accounting for how sensors' errors co-vary. Formula: $(x-\mu)^T \Sigma^{-1} (x-\mu)$.

Why this matters: switching from flat MSE to Mahalanobis was the change that finally moved F1 from ~0.48 to ~0.60. **What if you reverted to flat MSE?** You would lose the ability to say 'sensor 7 is off by 4 of its own standard deviations' and fall back to 'the average error is a bit high' — far less discriminative.

3.7 From window scores to per-timestep scores

The submission is per-timestep, but the model scores per-window. `expand_window_scores_to_timesteps` scores densely (stride 1) and averages every overlapping window covering a timestep. A single timestep near the middle of a run is covered by up to 200 windows; averaging them yields a smooth, robust per-timestep score instead of a coarse blindly-broadcast block label (the pre-fix behaviour).

3.8 Thresholding — F-beta = 0.5

A score is continuous; a prediction is 0/1. `find_best_threshold` sweeps the precision-recall curve and picks the cutoff maximising F-beta with $\beta=0.5$. $\beta < 1$ weights *precision* above recall. Why: the normal and anomalous score distributions overlap a lot, and plain F1 ($\beta=1$) kept choosing an aggressive high-recall threshold that flagged 65–88% of all timesteps — the project's chronic over-prediction symptom. $\beta=0.5$ trades some recall for a much saner false-positive rate.

IMPORTANT NUANCE (a live improvement question): the portal appears to score plain **F1**. You are optimising the threshold for F0.5. If the portal metric really is F1, a threshold tuned for F1 could score higher on the portal. This is worth testing — see the roadmap.

3.9 Evaluation

`compute_metrics` reports precision, recall, F1, ROC-AUC and the predicted anomaly rate. Note ROC-AUC is computed on the *raw scores* (threshold-free), so it measures the quality of the model's **ranking** — remember that number; Section 6 shows it is your true ceiling.

4. The CNN autoencoder, layer by layer

The input to the network is one window shaped **18 channels x 200 timesteps** (sensors are the convolution 'channels', time is the length). The encoder compresses; the decoder rebuilds. The width of each bar below is proportional to the time dimension, so you can literally see the $200 \rightarrow 100 \rightarrow 200$ hourglass.

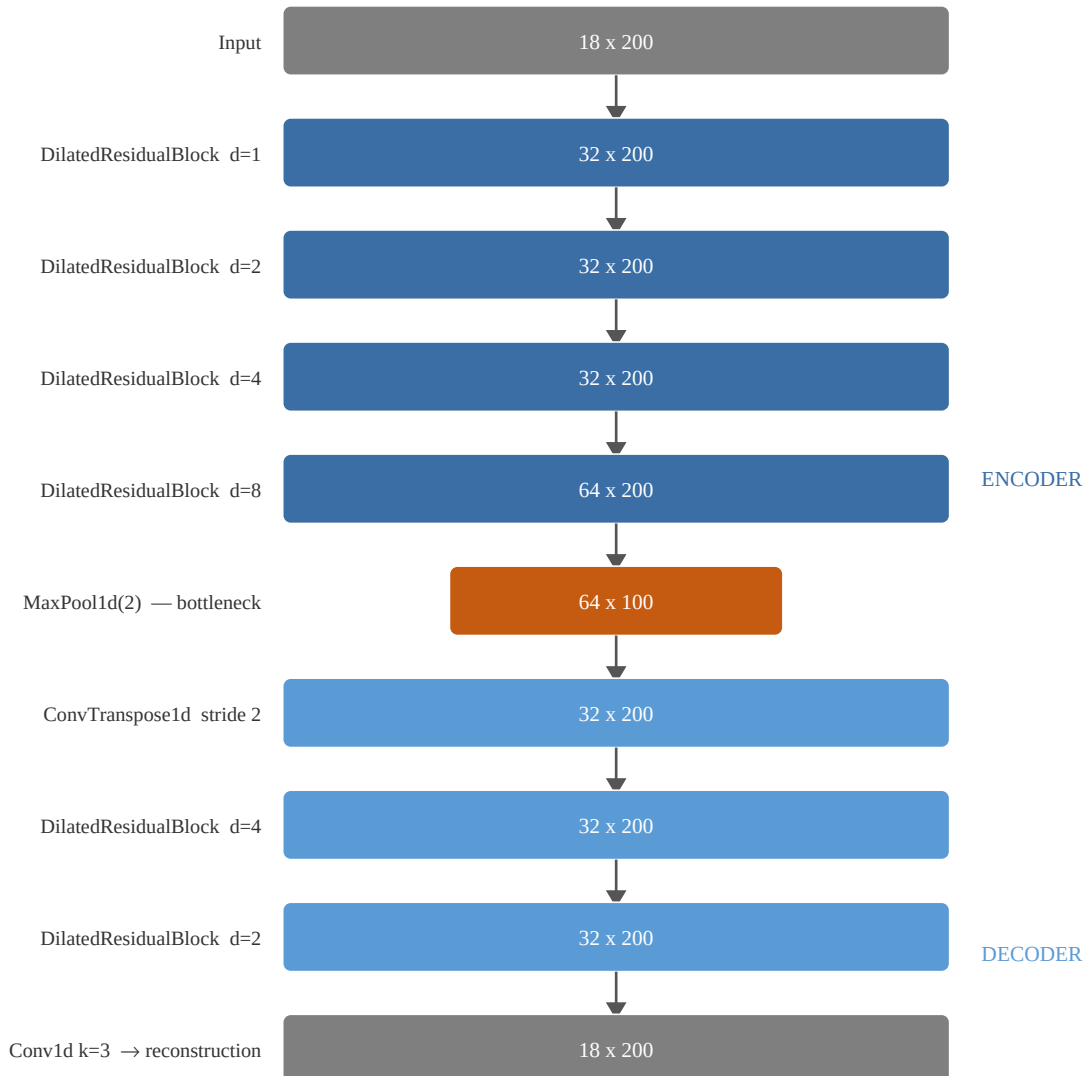


Figure 2. ConvAutoencoder1D. Only MaxPool1d halves time ($200 \rightarrow 100$); the dilated blocks use 'same' padding so they change channels, not length. ConvTranspose1d restores 200. The final Conv1d maps back to 18 sensors.

4.1 What a DilatedResidualBlock does

- **Two Conv1d layers** (kernel 3) with BatchNorm and ReLU. A convolution slides a small learned filter along time, detecting local temporal patterns shared across positions.
- **Dilation** inserts gaps between the filter taps. Dilation 8 means the filter looks at timesteps t , $t \pm 8$, $t \pm 16$ — covering a wide span with few parameters. Stacking dilations 1, 2, 4, 8 grows the **receptive field** to ~63 timesteps, so a neuron near the bottleneck 'sees' a long stretch of context. The old architecture was a single kernel-3 conv with a receptive field of just 3 timesteps — blind to the slow drifts that are the actual anomalies.

- **Residual (skip) connection:** the block computes $output = F(x) + x$ (a 1x1 conv projects x when the channel count changes). This lets gradients flow through deep stacks and lets each block learn a refinement rather than a full transform — the core idea behind ResNets/TCNs.
- **'Same' padding** keeps the time length fixed inside a block because this is reconstruction, not forecasting — the model may use both past and future context within the window (non-causal).

4.2 Why an hourglass (the bottleneck) is the whole point

MaxPool1d(2) squeezes time 200→100 at the narrowest point. This **bottleneck** forces the network to throw information away and keep only what is needed to rebuild normal windows. An autoencoder with no bottleneck could learn the identity function (copy input to output) and reconstruct *everything* perfectly — including anomalies — making the error score useless. The compression is what makes anomalies reconstruct badly. **What if the bottleneck were larger?** Easier reconstruction, smaller normal-vs-anomalous gap, weaker score. Too small? It cannot even rebuild normal data and everything looks anomalous. It is a capacity dial.

4.3 Data-flow table

Stage	Operation	Output (C x T)	Role
Input window	—	18 x 200	one scaled window, sensors as channels
Enc block d=1	conv+BN+ReLU x2, +skip	32 x 200	local temporal features
Enc block d=2	dilation 2	32 x 200	wider context
Enc block d=4	dilation 4	32 x 200	wider context
Enc block d=8	dilation 8, 32→64 ch	64 x 200	long-range context, richer channels
MaxPool1d(2)	downsample time	64 x 100	bottleneck — forces compression
ConvTranspose1d	upsample stride 2	32 x 200	begin reconstruction
Dec block d=4	dilation 4	32 x 200	refine
Dec block d=2	dilation 2	32 x 200	refine
Conv1d k=3	project to sensors	18 x 200	reconstructed window

The anomaly score is built from the gap between the Input row and the final row: per-sensor MSE between them, then Mahalanobis distance (Section 3.6).

5. Design choices, at a glance

Choice	Why	Effect / evidence
Autoencoder (not a classifier)	No anomaly labels exist to train a classifier	Simple way to learn from unlabeled normal data
Dilated TCN blocks	Anomalies are slow drifts; need wide receptive field	ROC-AUC ↑; F1 0.458 → 0.484
Bottleneck (MaxPool)	Prevent identity learning; force compression	Makes anomalies reconstruct badly
Per-run windowing	Runs are independent experiments	Removes fake cross-run transients
Per-sensor + Mahalanobis	Don't dilute one sensor's spike with another's	F1 ↑ 0.48 → 0.60 (biggest single jump)
Checkpoint selection	Val score peaks before training loss does	0.05 F1; avoids over-training
F-beta = 0.5 threshold	Overlapped distributions; plain F1 over-flags	Fixed 65–88% over-flagging
Dense scoring + averaging	Submission is per-timestep	Smooth, honest per-timestep signal

6. Where the F1 is actually lost (bottlenecks & failure modes)

Your best validation F1 is ~0.60 (portal 0.63) with ROC-AUC ~0.76. The gap between a perfect 1.0 and your 0.60 has a small number of concrete causes, in rough order of impact:

- 1 **The score-separability ceiling (the big one).** ROC-AUC ~0.76 means the model's ranking of anomalous vs normal timesteps genuinely overlaps. F1 at the best threshold is bounded by that overlap. No amount of threshold tuning breaks this ceiling — only a *better model / better score* raises AUC. This is why score improvements dominate threshold improvements.
- 2 **Training-set contamination.** The 'normal' training data almost certainly contains some undetected anomalies. The autoencoder slowly learns to reconstruct them, narrowing the normal-vs-anomalous error gap. Your own training curves (loss keeps dropping while val peaks at epoch ~45) are the fingerprint of this.
- 3 **Everything is tuned on the same 10-run validation set.** Window size, stride, architecture, checkpoint, Mahalanobis model, and threshold are all chosen by watching the same 10 runs. Your validation F1 is therefore an optimistic, 'seen' estimate. (Reassuringly, the portal 0.63 > val 0.60, so you are not badly over-fit — but you have no clean held-out estimate.)
- 4 **Threshold objective may not match the portal metric.** You optimise F0.5; the portal appears to score F1. That mismatch can leave portal F1 on the table (Section 3.8).
- 5 **Run-to-run variance.** Identical configs scored 0.568–0.611 purely from training stochasticity. A single run is a noisy estimate of the architecture's true quality.

7. How to think about improving this model

This section is about the *reasoning process*, not a shopping list. Internalise these five habits and you can generate and triage your own ideas.

7.1 Diagnose before you prescribe

Never change the model before you have *looked* at how it fails. Plot the Mahalanobis score over time for each validation run with the true anomaly regions shaded. Ask: are we **missing** anomalies (false negatives) or **over-flagging** normal stretches (false positives)? Histogram the scores for normal vs anomalous timesteps and look at the overlap. These two plots tell you whether your problem is the threshold, the score, or a specific anomaly type — and each implies a different fix.

7.2 Know your ceiling: separate ranking from thresholding

Two different quantities: ROC-AUC measures how well the model *rank*s anomalies (threshold-free); F1 measures a single operating point. If AUC is low, tuning the threshold is rearranging deck chairs — go improve the score. If AUC is high but F1 is low, *then* the threshold is your problem. Always ask which one you are actually fighting.

7.3 Match the objective to the metric you are graded on

If the portal scores F1, tuning for F0.5 is optimising the wrong thing. More generally: every place you 'pick the best X on validation' hides an objective (beta, the checkpoint metric, the fusion weight). Make sure that objective is the thing you ultimately care about.

7.4 Change one thing, measure honestly, respect variance

Because a single run varies by ± 0.02 F1, a change that 'improves' F1 by 0.01 has proven nothing. Change one variable at a time; where you can, average over seeds or use run-grouped cross-validation (you already built this in `run_final_hybrid.py`) to get an estimate that is not fooled by noise or by peeking at validation.

7.5 Attack the dominant bottleneck, not the convenient one

It is tempting to tweak the threshold because it is easy. But if the ceiling is score separability (Section 6.1), the high-leverage work is raising AUC: cleaner training signal, better normal model, better architecture. Spend effort where the bottleneck is, even when it is harder.

8. Prioritised improvement roadmap

Each item lists the intuition, the expected benefit, the risk, and what to study first. Ordered by expected payoff per unit effort.

1. Contamination-robust training (*attacks bottleneck 6.2*)

- **Intuition:** Down-weight the highest-loss training windows each batch — they are the likeliest undetected anomalies polluting your 'normal' set. Cleaner normal signal → bigger error gap → higher AUC.
- **Expected benefit:** Directly targets the dominant ceiling; already coded in your teammate's `train.py` (weight decay + early stopping + robust down-weighting).
- **Risk:** Down-weighting too aggressively discards hard-but-normal windows; needs an A/B against the current CNN.
- **Study first:** Robust statistics / trimmed loss; the M-ELBO idea from Xu et al. 2018 (Donut).

2. Seed-ensemble the CNN

- **Intuition:** Train 3–5 CNNs with different seeds and average their per-timestep scores. Ensembling cancels each model's idiosyncratic errors and typically raises AUC; it also removes the 0.568–0.611 run-to-run lottery.
- **Expected benefit:** Low-risk, reliable small gain and a much more stable number to report. Cheap on the V100.
- **Risk:** 5x training cost; scores must be normalised consistently before averaging.
- **Study first:** Ensemble methods; why variance reduction improves AUC.

3. Fit the Mahalanobis model on train, not validation

- **Intuition:** Currently μ /precision come from the few normal windows in the 10-run val set. Train has ~18k windows of (assumed) normal data — a far better-conditioned covariance estimate.
- **Expected benefit:** Better normal model → better Mahalanobis scores; also frees validation to be used only for the threshold.
- **Risk:** Train contamination biases μ ; mitigate by fitting on the lowest-error training windows only.
- **Study first:** Covariance estimation, Ledoit-Wolf shrinkage, Mahalanobis distance geometry.

4. Align the threshold to the portal metric

- **Intuition:** If the portal grades F1, choose the threshold that maximises validation F1 ($\beta=1$), or sweep β and submit the best. You may already be leaving portal F1 on the table.
- **Expected benefit:** Potentially free F1 with zero model change; a few minutes of work.
- **Risk:** You only get portal feedback per submission; verify the portal metric first to avoid guessing.
- **Study first:** Precision-recall trade-offs; the F-beta family and what β encodes.

5. Honest tuning split / nested CV

- **Intuition:** Reuse the run-grouped GroupKFold you already wrote so window size, threshold, and architecture are chosen without peeking at the data you report on.
- **Expected benefit:** A trustworthy generalisation estimate — stops silent over-fitting to 10 runs.
- **Risk:** Fewer runs per fold makes each estimate noisier; needs care with such a small val set.
- **Study first:** Cross-validation, data leakage, nested CV.

6. Architecture / window sweeps (last, not first)

- **Intuition:** Try window 300, a smaller/larger bottleneck, or LayerNorm instead of BatchNorm.
- **Expected benefit:** Possible AUC gains, but diminishing returns and easy to over-fit the search to validation.
- **Risk:** Highest risk-per-gain; only worthwhile after 1–4 and with an honest tuning split.
- **Study first:** Receptive-field arithmetic, normalization layers, the bias-variance trade-off.

Recommended order of attack: **diagnose (7.1)** → **robust training (1)** → **seed ensemble (2)** → **Mahalanobis-on-train (3)** → **threshold/metric alignment (4)**. Do 5 (honest CV) alongside so every result you trust is measured without peeking. Leave 6 for last.

A closing principle

The strongest version of this project is not the one with the most models — it is the one you understand well enough to explain every number. You have already shown this discipline: you tried fusion three ways and rejected it with evidence. Keep that habit. Raise AUC at the source, measure without fooling yourself, and let the threshold be the last small polish on a score you have earned.